ACCESS CONTROL AND AUTHENTICATION ASSIGNMENT

1. Create a simple login form for a mock Zomato admin panel that asks for username and password, then displays 'Access Granted' only if both fields match hardcoded admin credentials.

```
<!DOCTYPE html>
<html>
<head>
   <title>Zomato Admin Login</title>
</head>
<body>
   <h2>Zomato Admin Login</h2>

   <input type="text" id="username" placeholder="Username"><br><br>
   <input type="password" id="password" placeholder="Password"><br><br>

   <button onclick="checkLogin()">Login</button>
   <p id="result"></p>

   <script>
      function checkLogin() {
         let user = document.getElementById("username").value;
         let pass = document.getElementById("password").value;

         if (user === "admin" && pass === "zomato123") {
            document.getElementById("result").innerHTML = "Access Granted";
         } else {
            document.getElementById("result").innerHTML = "Access Denied";
         }
      }
   </script>
</body>
</html>
```

Explanation: The form compares the entered username and password with the hardcoded credentials (admin and zomato123). It displays 'Access Granted' only when both values match; otherwise, it displays 'Access Denied'.

2. Write a Python function check_access(user_role, resource) that returns True if the user_role is allowed to access the resource, otherwise False. Test it with roles like 'customer', 'restaurant_owner', and resources like 'menu_edit', 'order_history', 'admin_dashboard'.

```
def check_access(user_role, resource):
   permissions = {
      "customer": ["order_history"],
      "restaurant_owner": ["menu_edit", "order_history"],
      "admin": ["menu_edit", "order_history", "admin_dashboard"]
   }
```

```
    return resource in permissions.get(user_role, [])


# Testing the function
print(check_access("customer", "order_history"))
print(check_access("customer", "admin_dashboard"))
print(check_access("restaurant_owner", "menu_edit"))
print(check_access("restaurant_owner", "admin_dashboard"))
```

Output: True, False, True, False

Explanation: The dictionary maps each role to a list of permitted resources. The function checks whether the requested resource is in that role's list and returns True or False.

3. Simulate a two-factor authentication flow like Paytm: after a user enters the correct password, generate a 6-digit OTP and display it in the console. Ask the user to input the OTP and display 'Login Successful' only if it matches.

```
import random

correct_password = "paytm123"
password = input("Enter password: ")

if password == correct_password:
    otp = random.randint(100000, 999999)
    print("Your OTP is:", otp)

    user_otp = input("Enter OTP: ")

    if user_otp == str(otp):
        print("Login Successful")
    else:
        print("Invalid OTP")
else:
    print("Incorrect Password")
```

Explanation: The program first checks the password. If it is correct, it generates and displays a random six-digit OTP in the console. The user must enter the same OTP to complete the login.

4. Given an array of user objects with properties: username, isPremium, and lastLogin, write a function that returns only those users who are premium and have logged in within the last 7 days. Assume lastLogin is in YYYY-MM-DD format and today is 2024-06-10.

```
function getPremiumUsers(users) {
    const today = new Date("2024-06-10");
    const sevenDaysAgo = new Date("2024-06-03");

    return users.filter(user => {
        const loginDate = new Date(user.lastLogin);

        return user.isPremium &&
               loginDate >= sevenDaysAgo &&
               loginDate <= today;
```

```
    });
}

const users = [
    { username: "Rahul", isPremium: true, lastLogin: "2024-06-08" },
    { username: "Amit", isPremium: false, lastLogin: "2024-06-09" },
    { username: "Priya", isPremium: true, lastLogin: "2024-06-01" },
    { username: "Neha", isPremium: true, lastLogin: "2024-06-05" }
];

console.log(getPremiumUsers(users));
```

Output: Rahul and Neha are returned because both are premium users and their last login dates fall between 3 June and 10 June 2024.

Explanation: The filter() method keeps only users whose isPremium value is true and whose lastLogin date is within the specified seven-day period.

5. Use ChatGPT to generate a list of 5 common access control mistakes developers make in real-world apps like Instagram or Flipkart, and for each, write a one-line fix or prevention tip.

Access control mistake	Prevention tip
1. Giving users unnecessary permissions	Follow the principle of least privilege and give each user only the permissions needed.
2. Not checking user roles	Verify the user's role and permissions before allowing access to a resource.
3. Exposing admin pages to normal users	Restrict administrative pages and functions to authorized administrators.
4. Insecure direct object references	Check that the logged-in user is authorized to access the specific record or object requested.
5. Poor session management	Use secure session cookies, expire inactive sessions, and invalidate sessions at logout.

Conclusion: Access control ensures that users can access only the information and functions they are authorized to use. Authentication verifies a user's identity, while authorization determines what that user is allowed to do.